Expressions: https://www.postgresql.org/docs/current/sql-expressions.html
Functions and operators: https://www.postgresql.org/docs/current/functions.html

*** Syntax**
Lexical structure: https://www.postgresql.org/docs/current/sql-syntax-lexical.html

```
-- standard SQL comment

/*
  C style
  multiline comment
 */



---- System
--
current_catalog           -- current DB
current_database()        -- same

current_schema            -- current active schema i.e. first match
in search_path
current_schema()
current_schemas(TRUE)     -- schemas active in search path
pb_my_temp_schema()       -- OID of temp schema

current_role              -- current user
current_user              -- same
user
session_user
system_user

current_query()
```

*** Casting**
Expressions: https://www.postgresql.org/docs/current/sql-expressions.html

```
CAST (x AS INTEGER)       -- SQL
x::INTEGER                -- PostgreSQL
```

*** Array functions and operators**
Array functions: https://www.postgresql.org/docs/current/functions-array.html

```
-- operations
anyarray @> anyarray → boolean
anyarray <@ anyarray → boolean
anyarray && anyarray → boolean
```

```
anycompatiblearray || anycompatiblearray → anycompatiblearray
             -- concat
anycompatible || anycompatiblearray → anycompatiblearray
anycompatiblearray || anycompatible → anycompatiblearray



-- array info
array_position ( anycompatiblearray, anycompatible [, integer ] ) →
integer
array_positions ( anycompatiblearray, anycompatible ) → integer[]
cardinality ( anyarray ) → integer
array_dims ( anyarray ) → text
array_ndims ( anyarray ) → integer
array_length ( anyarray, integer ) → integer
array_lower ( anyarray, integer ) → integer
array_upper ( anyarray, integer ) → integer

-- array manipulation
array_cat ( anycompatiblearray, anycompatiblearray ) →
anycompatiblearray
array_append ( anycompatiblearray, anycompatible ) →
anycompatiblearray
array_prepend ( anycompatible, anycompatiblearray ) →
anycompatiblearray
array_remove ( anycompatiblearray, anycompatible ) →
anycompatiblearray
array_replace ( anycompatiblearray, anycompatible, anycompatible )
→ anycompatiblearray
trim_array ( array anyarray, n integer ) → anyarray
array_fill ( anyelement, integer[] [, integer[] ] ) → anyarray



-- others
array_to_string ( array anyarray, delimiter text [, null_string
text ] ) → text
unnest ( anyarray ) → setof anyelement
unnest ( anyarray, anyarray [, ... ] ) → setof anyelement,
anyelement [, ... ]
```

```
-- Functions
array_position(Array[1,2,3,1],2)      -- 2  First position
array_positions(Array[1,2,3,2],2)     -- [2,4]
array_append(Array[1,2,3], 4)         -- [1,2,3,4]
array_prepend(4, Array[1,2,3])        -- [4, 1,2,3]
array_remove(Array[1,2,3], 1)         -- [2,3]
array_replace(Array[1,2,3], 1, 10)    -- [10,2,3]
array_fill(..)

array_to_string(Array[1,2,3], '|')    -- '1|2|3'
array_shuffle(...)
array_sample(...)
```

```
...


-- Operators
Array[1,2,3] @> Array[1,1,2,2]         -- [1,2,3] contains 1,2?
TRUE
Array[1,1,2,2] <@ Array[1,2,3]         -- reverse
Array[1,4,5] && Array[1,2,3]           -- non-empty intersection?
TRUE
ARRAY[1,2,3] || ARRAY[4,5,6]           -- concat
ARRAY[1,2,3,4,5,6]
ARRAY[1,2,3] || 4                      -- append  ARRAY[1,2,3,4]
4 || ARRAY[1,2,3]                      -- prepend


-- ALL/SOME/ANY
--
-- Same as subquery ALL/SOME for arrays
10 > ALL (Array[1,2,3])
10 > ALL Array[1,2,3]          -- ERROR. Need brackets
2 > SOME (Array[1,2,3])
1 > ANY (Array[1,2,3])         -- same
1 = SOME (Array[1,2,3])        -- equality
```

*** data formatting**
https://www.postgresql.org/docs/current/functions-formatting.html

```
-- to text
to_char ( timestamp, text ) → text
to_char ( timestamp with time zone, text ) → text
to_char ( interval, text ) → text
to_char ( numeric_type, text ) → text


-- from text
to_date ( text, text ) → date
to_number ( text, text ) → numeric
to_timestamp ( text, text ) → timestamp with time zone
```

*** date**
https://www.postgresql.org/docs/current/functions-datetime.html

```
EXTRACT(day FROM '2024-01-11'::date)       -- 11
EXTRACT(month FROM '2024-01-11'::date)     -- 1
EXTRACT(year FROM '2024-01-11'::date)      -- 2024
EXTRACT(week FROM '2024-11-10'::date)      -- 42
```

```
EXTRACT(dow FROM '2024-11-10'::date)       -- 0=sun 6=sat
EXTRACT(isodow FROM '2024-11-10'::date)    -- 1=mon 7=sun


-- date_trunc: round date down to a given precision
--
-- e.g. 'month' rounds down to first day of month
date_trunc('year', '2025-11-19'::date)     -- '2025-01-01 00:00:00'
date_trunc('month', '2025-11-19'::date)    -- '2025-11-01 00:00:00'
date_trunc('week', '2025-11-19'::date)     -- '2025-11-17 00:00:00'
Monday of week
date_trunc('day', '2025-11-19'::date)      -- '2025-11-19
00:00:00'
```

*** system functions**
https://www.postgresql.org/docs/current/functions-info.html

```
-- types
format_type ( type oid, typemod integer ) → text
pg_basetype ( regtype ) → regtype
pg_basetype('mytext'::regtype) → text
pg_char_to_encoding ( encoding name ) → integer
pg_encoding_to_char ( encoding integer ) → name
pg_get_expr ( expr pg_node_tree, relation oid [, pretty boolean ] )
→ text
pg_typeof ( "any" ) → regtype
                  -- e.g. pg_typeof(1::int) = 'int'

                          pg_typeof('abc'::text) = 'text'
COLLATION FOR ( "any" ) → text


-- get OID from name
to_regclass ( text ) → regclass
to_regcollation ( text ) → regcollation
to_regnamespace ( text ) → regnamespace
to_regoper ( text ) → regoper
to_regoperator ( text ) → regoperator
to_regproc ( text ) → regproc
to_regprocedure ( text ) → regprocedure
to_regrole ( text ) → regrole
to_regtype ( text ) → regtype                         -- e.g.
to_regtype('text') = 25 = oid of text
to_regtypemod ( text ) → integer


-- Functions
pg_get_functiondef ( func oid ) → text
pg_get_function_arguments ( func oid ) → text
pg_get_function_identity_arguments ( func oid ) → text
```

```
pg_get_function_result ( func oid ) → text


-- Table
pg_get_partkeydef ( table oid ) → text
pg_get_catalog_foreign_keys () → setof record ( fktable regclass,
fkcols text[], pktable regclass, pkcols text[], is_array boolean,
is_opt boolean )
pg_get_constraintdef ( constraint oid [, pretty boolean ] ) → text
pg_options_to_table ( options_array text[] ) → setof record
( option_name text, option_value text )
pg_get_serial_sequence ( table text, column text ) → text


-- Index
pg_get_indexdef ( index oid [, column integer, pretty boolean ] ) →
text
pg_index_column_has_property ( index regclass, column integer,
property text ) → boolean
pg_index_has_property ( index regclass, property text ) → boolean
pg_indexam_has_property ( am oid, property text ) → boolean


-- View
pg_get_viewdef ( view oid [, pretty boolean ] ) → text
pg_get_viewdef ( view oid, wrap_column integer ) → text
pg_get_viewdef ( view text [, pretty boolean ] ) → text



-- Other objects
pg_get_ruledef ( rule oid [, pretty boolean ] ) → text
pg_get_statisticsobjdef ( statobj oid ) → text
pg_get_triggerdef ( trigger oid [, pretty boolean ] ) → text


-- User
pg_get_userbyid ( role oid ) → name



-- Tablespace
pg_tablespace_databases ( tablespace oid ) → setof oid
pg_tablespace_location ( tablespace oid ) → text



-- Misc
pg_get_keywords () → setof record ( word text, catcode "char",
barelabel boolean, catdesc text, baredesc text )
pg_settings_get_flags ( guc text ) → text[]
```

==========

== strings ==

Ref: https://www.postgresql.org/docs/current/functions-string.html

```
-- conversion
ascii ( text ) → integer                                        -- e.g.
ascii('a') = 97
chr ( integer ) → text
to_ascii ( string text ) → text
to_ascii ( string text, encoding integer ) → text
to_bin ( integer ) → text
to_bin ( bigint ) → text
to_hex ( integer ) → text
to_hex ( bigint ) → text
to_oct ( integer ) → text
to_oct ( bigint ) → text
unistr ( text ) → text


-- basic
length ( text ) → integer
strpos ( string text, substring text ) → integer
md5 ( text ) → text
repeat ( string text, number integer ) → text
starts_with ( string text, prefix text ) → boolean


-- transform
concat ( val1 "any" [, val2 "any" [, ...] ] ) → text
reverse ( text ) → text
translate ( string text, from text, to text ) → text
concat_ws ( sep text, val1 "any" [, val2 "any" [, ...] ] ) → text


-- formatting
format ( formatstr text [, formatarg "any" [, ...] ] ) → text
initcap ( text ) → text
                -- capitalize e.g. initcap('abc DEF') = 'Abc Def'
replace ( string text, from text, to text ) → text


-- substring
substr ( string text, start integer [, count integer ] ) → text
left ( string text, n integer ) → text
right ( string text, n integer ) → text


-- split
split_part ( string text, delimiter text, n integer ) → text
                    -- e.g. split_part('a_b_c', '_', 1) = 'a'
```

```
                      --      split_part('a_b_c', '_', 2) = 'b'
string_to_array ( string text, delimiter text [, null_string
text ] ) → text[]
string_to_table ( string text, delimiter text [, null_string
text ] ) → setof text


-- regexp
regexp_count ( string text, pattern text [, start integer [, flags
text ] ] ) → integer
regexp_instr ( string text, pattern text [, start integer [, N
integer [, endoption integer [, flags text [, subexpr
integer ] ] ] ] ] ) → integer
regexp_like ( string text, pattern text [, flags text ] ) → boolean
regexp_match ( string text, pattern text [, flags text ] ) → text[]
regexp_matches ( string text, pattern text [, flags text ] ) →
setof text[]
regexp_replace ( string text, pattern text, replacement text [,
start integer ] [, flags text ] ) → text
regexp_replace ( string text, pattern text, replacement text, start
integer, N integer [, flags text ] ) → text
regexp_split_to_array ( string text, pattern text [, flags text ] )
→ text[]
regexp_substr ( string text, pattern text [, start integer [, N
integer [, flags text [, subexpr integer ] ] ] ] ) → text


-- SQL
pg_client_encoding ( ) → name
parse_ident ( qualified_identifier text [, strict_mode boolean
DEFAULT true ] ) → text[]
quote_ident ( text ) → text
quote_literal ( text ) → text
quote_literal ( anyelement ) → text
quote_nullable ( text ) → text
quote_nullable ( anyelement ) → text
```

*** Basic**

Lexical structure: https://www.postgresql.org/docs/current/sql-syntax-lexical.html

```
-- string literal
'abc'
'a                -- multiline
b
c
d'

E'\n'             -- newline
E'\t'
```

```
-- dollar-quoted
$$ab                  -- multiline
c
d
$$
$$a'bc'd$$               -- "a'bc'd"  no need to escape single
quote
$mytag$abcd$mytag$       -- "abcd"    with tags


-- Escapes
'\141'                -- BAD
E'a'                  -- a
E'\141'               -- a in octal
E'\x61'               -- a in hex
E'\u0061'             -- a in unicode
```

*** Pattern matching**
Ref: https://www.postgresql.org/docs/current/functions-matching.html

Pattern matching
String patterns.
Postgres has 3 types of pattern matching expressions

LIKE	SQL standard
NOT LIKE	Must match **whole** text
SIMILAR TO	PostgreSQL regular expression
NOT SIMILAR TO	Must match **whole** text
~	POSIX regular expression
~* (case insensitive)	
!~	
!~*	

e.g.

```
-- match
SELECT 'abc' LIKE 'ab_';          -- TRUE
SELECT 'abc' SIMILAR TO 'ab_';    -- TRUE
SELECT 'abc' ~ 'ab.';             -- TRUE

-- whole/partial match
SELECT 'abc' LIKE 'a';            -- FALSE
SELECT 'abc' SIMILAR TO 'a';      -- FALSE
SELECT 'abc' ~ 'a';               -- TRUE

-- e.g. wildcard
SELECT 'abc' LIKE 'a%';
SELECT 'abc' SIMILAR TO 'a%';
SELECT 'abc' ~ 'a.+';
```

```
# SIMILAR TO
%                 same as LIKE
_
a|b               OR
a*
a+
a?                0 or 1
a{2}              exactly 2
a{2,}             2 or more
a{2,5}
(a|b)+            group with ()
[a-zA-Z0-9]

# escapes
\|
\*
\+
\{
\+
\?
```

*** Regular expression**
Ref: https://www.postgresql.org/docs/current/functions-matching.html

```
regexp_match('123124125', '(12)(.)')               -- text[] {12,
3}
regexp_matches('123124125', '(12)(.)', 'g')        -- setof text[]
{12, 3} {12, 4} {12, 5}
regexp_matches('123124125', '(12)(.)')             -- one row only
```

*** format()**
Ref: https://www.postgresql.org/docs/current/functions-string.html

format() function
Similar to **sprintf()**

Syntax

%[n$][[-]w]type

where

type = s | I | L
s = normal string
I = SQL identifier
i.e. double quote as needed
L = SQL string literal
i.e. single quote as needed

n = position

w = max width
-w = left aligned max width

```
format('value is %s', 'abc')      -- 'value is abc'
format('value is %s', 1)          -- 'value is 1'

format('value is %s', NULL)       -- '' i.e. empty string
format('value is %I', 'ab c')     -- 'value is "ab c"'
format('value is %I', 1)          -- 'value is "1"
format('value is %I', NULL)       -- ERROR

format('value is %L', 'abc')      -- 'value is 'abc''
format('value is %L', 'abc')      -- 'value is '1''
format('value is %L', NULL)

-- position: [n]%
format('%1$s is %2$s', 'abc', 'def')      -- 'abc is def'
format('%s is %s', 'abc', 'def')          -- same
format('%2$s is %1$s', 'abc', 'def')      -- 'def is abc'


-- width
format('value is |%5s|', 'abc')           -- width 5       e.g.
'value is |  abc|'
format('value is |%1s|', 'abc')           -- ignored if too small
e.g. 'value is |abc|'
format('value is |%-5s|', 'abc')          -- left align
e.g.'value is |abc|'
format('value is |%*s|', 5, 'abc')        -- same from parameter
format('value is |%2$*1$s|', 'abc', 5)    -- same with parameter
positions
```

<u>* string functions</u>

SQL standard

```
-- Query
char_length ( text ) → integer
character_length ( text ) → integer
octet_length ( text ) → integer
octet_length ( character ) → integer
position ( substring text IN string text ) → integer

-- Formating
lower ( text ) → text
upper ( text ) → text

trim ( [ LEADING | TRAILING | BOTH ] [ characters text ] FROM
string text ) → text
trim ( [ LEADING | TRAILING | BOTH ] [ FROM ] string text [,
```

```
characters text ] ) → text
rpad ( string text, length integer [, fill text ] ) → text
lpad ( string text, length integer [, fill text ] ) → text
rtrim ( string text [, characters text ] ) → text
ltrim ( string text [, characters text ] ) → text


-- Substring
substring ( string text [ FROM start integer ] [ FOR count
integer ] ) → text
substring ( string text FROM pattern text ) → text
substring ( string text SIMILAR pattern text ESCAPE escape text ) →
text
substring ( string text FROM pattern text FOR escape text ) → text
overlay ( string text PLACING newsubstring text FROM start integer
[ FOR count integer ] ) → text

-- Unicode
normalize ( text [, form ] ) → text
unicode_assigned ( text ) → boolean
```

Non-standard SQL

```
text ^@ text → boolean

-- Query
pg_client_encoding ( ) → name
length ( text ) → integer
starts_with ( string text, prefix text ) → boolean
strpos ( string text, substring text ) → integer


-- Conversion
ascii ( text ) → integer
chr ( integer ) → text
to_ascii ( string text ) → text
to_ascii ( string text, encoding name ) → text
to_ascii ( string text, encoding integer ) → text
to_bin ( integer ) → text
to_bin ( bigint ) → text
to_hex ( integer ) → text
to_hex ( bigint ) → text
to_oct ( integer ) → text
to_oct ( bigint ) → text


-- Utils
unistr ( text ) → text
md5 ( text ) → text
repeat ( string text, number integer ) → text
```

```
-- Formatting
initcap ( text ) → text
left ( string text, n integer ) → text
right ( string text, n integer ) → text
substr ( string text, start integer [, count integer ] ) → text

format ( formatter text [, formatarg "any" [, ...] ] ) → text
replace ( string text, from text, to text ) → text
translate ( string text, from text, to text ) → text

split_part ( string text, delimiter text, n integer ) → text
string_to_array ( string text, delimiter text [, null_string
text ] ) → text[]
string_to_table ( string text, delimiter text [, null_string
text ] ) → setof text

concat ( val1 "any" [, val2 "any" [, ...] ] ) → text
concat_ws ( sep text, val1 "any" [, val2 "any" [, ...] ] ) → text
reverse ( text ) → text


-- SQL
parse_ident ( qualified_identifier text [, strict_mode boolean
DEFAULT true ] ) → text[]
quote_ident ( text ) → text
quote_literal ( text ) → text
quote_literal ( anyelement ) → text
quote_nullable ( text ) → text
quote_nullable ( anyelement ) → text


-- Regexp
regexp_count ( string text, pattern text [, start integer [, flags
text ] ] ) → integer
regexp_instr ( string text, pattern text [, start integer [, N
integer [, endoption integer [, flags text [, subexpr
integer ] ] ] ] ] ) → integer
regexp_like ( string text, pattern text [, flags text ] ) → boolean
regexp_match ( string text, pattern text [, flags text ] ) → text[]
regexp_matches ( string text, pattern text [, flags text ] ) →
setof text[]
regexp_replace ( string text, pattern text, replacement text [,
start integer ] [, flags text ] ) → text
regexp_replace ( string text, pattern text, replacement text, start
integer, N integer [, flags text ] ) → text
regexp_split_to_array ( string text, pattern text [, flags text ] )
→ text[]
regexp_split_to_table ( string text, pattern text [, flags text ] )
→ setof text
regexp_substr ( string text, pattern text [, start integer [, N
integer [, flags text [, subexpr integer ] ] ] ] ) → text
```

==========

== aggregate/window functions ==

Expressions: https://www.postgresql.org/docs/current/sql-expressions.html

Aggregate function

```
aggregate_name (expression [ , ... ] [ order_by_clause ] ) [ FILTER
( WHERE filter_clause ) ]
aggregate_name (ALL expression [ , ... ] [ order_by_clause ] )
[ FILTER ( WHERE filter_clause ) ]
aggregate_name (DISTINCT expression [ , ... ] [ order_by_clause ] )
[ FILTER ( WHERE filter_clause ) ]
aggregate_name ( * ) [ FILTER ( WHERE filter_clause ) ]
aggregate_name ( [ expression [ , ... ] ] ) WITHIN GROUP
( order_by_clause ) [ FILTER ( WHERE filter_clause ) ]
```

Window function

```
function_name ([expression [, expression ... ]]) [ FILTER ( WHERE
filter_clause ) ] OVER window_name
function_name ([expression [, expression ... ]]) [ FILTER ( WHERE
filter_clause ) ] OVER ( window_definition )
function_name ( * ) [ FILTER ( WHERE filter_clause ) ] OVER
window_name
function_name ( * ) [ FILTER ( WHERE filter_clause ) ] OVER
( window_definition )

window_definition =
[ existing_window_name ]
[ PARTITION BY expression [, ...] ]
[ ORDER BY expression [ ASC | DESC | USING operator ] [ NULLS
{ FIRST | LAST } ] [, ...] ]
[ frame_clause ]

frame_clause =
{ RANGE | ROWS | GROUPS } frame_start [ frame_exclusion ]
{ RANGE | ROWS | GROUPS } BETWEEN frame_start AND frame_end
[ frame_exclusion ]

frame_start, frame_end =
UNBOUNDED PRECEDING
offset PRECEDING
CURRENT ROW
offset FOLLOWING
UNBOUNDED FOLLOWING

frame_exclusion =
EXCLUDE CURRENT ROW
```

```
EXCLUDE GROUP
EXCLUDE TIES
EXCLUDE NO OTHERS
```

```
SELECT

  -- string_agg
  string_agg(name, E',')
  string_agg(name, E',' ORDER BY id)       -- ORDER BY
FROM t
```

==========

== pg_catalog==

System catalog: https://www.postgresql.org/docs/current/catalogs.html
System views: https://www.postgresql.org/docs/current/views.html

pg_catalog
information_schema
pg_catalog tables are the source of **truth**.
pg_catalog views are human friendly views of the tables.
information_schema is derived from pg_catalog for SQL standard compatibility.

See

```
> \dtS+ pg_*     list tables e.g.
> \dvS+ pg_*     list views e.g. pg_authid
> \dS+           list all views and tables
```

*** All tables**
Ref: https://www.postgresql.org/docs/current/catalogs-overview.html

# Database	
pg_database	databases
pg_namespace	schemas
pg_class	tables, indexes, sequences, views (“relations”)
pg_attribute	table columns (“attributes”)
pg_attrdef	column default values
pg_inherits	table inheritance hierarchy
pg_partitioned_table	partition key of tables
pg_description	descriptions or comments on database objects
pg_tablespace	tablespaces within this database cluster
pg_trigger	triggers
pg_sequence	sequences
pg_event_trigger	event triggers
# Data	
pg_depend	dependencies between database objects
pg_shdepend	dependencies on shared objects
pg_shdescription	comments on shared objects
pg_shseclabel	security labels on shared database objects
pg_largeobject	data pages for large objects

pg_largeobject_metadata	metadata for large objects

# Data type	
pg_type	data types
pg_range	range types
pg_enum	enum label and value definitions
pg_transform	transforms (data type to procedural language conversions)
pg_cast	casts (data type conversions)
pg_collation	collations (locale information)
pg_conversion	encoding conversion information
pg_constraint	check constraints, unique constraints, primary key constraints, foreign key constraints

# Indexes	
pg_index	index information e.g. whether an index is a primary key
pg_opclass	access method operator classes
pg_operator	operators
pg_opfamily	access method operator families

# Functions	
pg_proc	functions and procedures
pg_aggregate	aggregate functions
pg_language	languages for writing functions

# Roles	
pg_authid	roles (authid=authorization identifiers)
pg_auth_members	role membership

# Priviliges	
pg_default_acl	default privileges for object types
pg_db_role_setting	per-role and per-database settings
pg_init_privs	object initial privileges
pg_parameter_acl	configuration parameters for which privileges have been granted
pg_policy	row-security policies
pg_seclabel	security labels on database objects
pg_rewrite	query rewrite rules

# Stats	
pg_statistic	planner statistics
pg_statistic_ext	CREATE STATISTICS
pg_statistic_ext_data	extended stats data

# Text search	
pg_ts_config	text search configurations
pg_ts_config_map	text search configurations' token mappings
pg_ts_dict	text search dictionaries
pg_ts_parser	text search parsers
pg_ts_template	text search templates

# Access methods	
pg_am	relation access methods
pg_amop	access method operators
pg_amproc	access method support functions

Replication

pg_publication	publications for logical replication
pg_publication_namespace	schema to publication mapping
pg_publication_rel	relation to publication mapping
pg_replication_origin	registered replication origins
pg_subscription	logical replication subscriptions
pg_subscription_rel	relation state for subscriptions

Misc

pg_extension	installed extensions
pg_foreign_data_wrapper	foreign-data wrapper definitions
pg_foreign_server	foreign server definitions
pg_foreign_table	additional foreign table information
pg_user_mapping	mappings of users to foreign servers

*** Data tables i.e. tables,**

```
-- Namespace
pg_ namespace.oid
pg_ namespace.nspname
pg_ namespace.owner
...


-- Data type
pg_type.oid
pg_type.typname
pg_type.typnamespace          Namespace [pg_namespace.oid]
pg_type.typowner              Owner [pg_authid.oid]
pg_type.typtype               b=base type
                              c=composite
                              d=domain
                              e=enum
                              p=pseudo
                              r=range
                              m=multirange
...


-- Comments
pg_description.objoid         [pg_class.oid]
pg_description.classoid
pg_description.objsubid       0 for table comment
                              attnum for table column comment
pg_description.description


-- Table of tables and table-like objects e.g. views
pg_class
pg_class.oid
pg_class.relname              Table name
pg_class.relnamespace         Table namespace [pg_namespace.oid]
pg_class.reltype              Table type [pg_type.oid]
```

```
pg_class.relowner             Table owner [pg_authid.oid]

-- storage
pg_class.relam
pg_class.relfilenode
pg_class.relpages
pg_class.reloptions           storage parameters
                              NULL = system default
pg_class.relallvisible
pg_class.reltuples            number of tuples if table. -1 if
unknown


-- properties
pg_class.relkind              r = table
                              i = index
                              s = sequence
                              t = TOAST
                              v = view
                              m = materialized view
                              c = composite type
                              f = foreign table
                              p = partitioned table i.e. the root
table
                              I = partitioned index
pg_class.relpersistence       p = permanent u =unlogged t=temporary
pg_class.reltablenamespace    0 = none [pg_tablespace.oid]
pg_class.reltoastrelid        0 = none [pg_class.oid]
pg_class.reloftype            associated type, if composite table
type [pg_type.oid]
pg_class.relnatts             # of columns
pg_class.relcheck             # of CHECK constraints
pg_class.relisshared          TRUE if shared in all databases i.e.
only system classes, like pg_class
pg_class.relispartition       TRUE if table partition i.e. the
child tables
pg_class.relhasindex
pg_class.relhassubclass       TRUE if INHERITED
pg_class.relreplindent


-- misc
pg_class.relhasrules
pg_class.relhastriggers
relrowsecurity
relforcesecurity


-- Table columns
pg_attribute
pg_attribute.attrelid         Table [pg_class.oid]
pg_attribute.atttypid         Data type of column [pg_type.oid]
pg_attribute.attname          Column name
```

```
pg_attribute.attnum           Column number
pg_attrribute.attnotnull
pg_attribute.hasdef
...
```

```
-- compute type name
-- e.g.  varchar(10)
         numeric(10, 2)
format_type(atttypeid, atttypmod)
```

```
-- DEFAUlT values
pg_attrdef
pg_attrdef.oid                Table [pg_class.oid]
pg_attrdef.adrelid            Column number [pg_attrbute.attnum]
pg_attrdef.adbin              Default value expression tree.
                              Convert to SQL expression

                                pg_get_expr(adbin, adrelid)



-- INDEX
pg_index.indexrelid           index [pg_class.oid]
pg_index.indrelid             table [pg_class.oid]
pg_index.indkey               column numbers, including INCLUDING
columns [array of pg_attribute.attnum]
                              indkey[:indkeyatts]       key columns
                              indkey[indkeyatts+1:]     INCLUDING
columns
pg_index.indkeyatts           number of columns, excluding
INCLUDING indexes (covering indexes)
pg_index.indnatts             number of columns, including
INCLUDING indexes
pg_index.indisunique          UNIQUE
pg_index.indisprimary         PRIMARY KEY
```

```
-- e.g. get index column names per table
SELECT
  indrelid,                   -- table
  indexrelid,                 -- index
  array(
    SELECT attname
    -- key columns only
    FROM unnest(indkey[:indnkeyatts]) t(i)
    INNER JOIN pg_attribute
    ON attrelid = indrelid AND attnum = i
    ORDER BY array_position(indkey, i) ASC
  )
FROM pg_index



-- approach 2: with GROUP BY
SELECT
```

```
  indrelid,                   -- table
  indexrelid,                 -- index
  array_agg(attname)          -- column names
FROM pg_index
INNER JOIN pg_attribute
ON indrelid = attrelid
AND attnum=any(indkey[:indnkeyatts])           -- key columns only
GROUP BY indrelid, indexrelid



-- CONSTRAINTs
pg_constraint
pg_constraint.name
pg_constraint.type            'c'  CHECK
                              'f'  FOREIGN KEY
                              'n'  NOT NULL
                              'p'  PRIMARY KEY
                              'u'  UNIQUE
                              't' trigger
                              'x'  EXCLUDE
pg_constraint.contypeid       pg_type.oid if a DOMAIN constraint
pg_constraint.relid           pg_class.oid if a table constraint
pg_constraint.conindid        pg_class.oid if associated with an
index e.g. UNIQUE, PRIMARY KEY, EXCLUDE
pg_constraint.parentid        pg_class.oid if partition constraint


-- pg_inherits
pg_inherits.inhparent         parent
pg_inherits.inhrelid          child


-- Routines i.e. FUNCTION/PROCEDURE
pg_proc.oid
pg_proc.proname
pg_proc.pronamespace
pg_proc.prokind char              f=normal p=procedure a=aggregate
w=window
pg_proc.pronargs
pg_proc.proargnames text[]
pg_proc.proargtypes oid[]         [pg_type.oid]
pg_proc.proargmodes char[]        i=IN o=OUT b=INOUT v=VARIADIC
t=TABLE
pg_proc.prorettype oid            [pg_type.oid]
....etc

-- get function info
pg_get_functiondef(oid)                        -- full function
body
pg_get_function_arguments(oid)
pg_get_function_return(oid)
```

```
pg_get_function_identity_arguments(oid)
```

*** Admin tables/views**

```
-- ROLE info
pg_authid.oid
pg_authid.rolname
pg_authid.rolconfig
pg_authid.rolvaliduntil       timestampz
-- privilege flags
pg_authid.rolsuper            is SUPERUSER
pg_authid.createrole          is CREATEROLE
pg_authid.createdb
...
```

```
-[ RECORD 1 ]--+-----------------------------
rolname        | yuinishizawa
rolsuper       | t
rolinherit     | t
rolcreaterole  | t
rolcreatedb    | t
rolcanlogin    | t
rolreplication | t
rolconnlimit   | -1
rolpassword    | ********
rolvaliduntil  |
rolbypassrls   | t
rolconfig      | {transaction_timeout=33s}
oid            | 10
```

```
-- ROLE membership info
pg_auth_members.oid
pg_auth_members.roleid                    rolid is in member
[pg_roldes.oid]
pg_auth_members.member                [pg_roldes.oid]
pg_auth_members.grantor
pg_auth_members.admin_option          ADMIN option
pg_auth_members.inherit_option        INHERIT option
pg_auth_members.set_option
```

```
-- Compiler configurations
--
-- e.g.  CC, CFLAG, LDFLAGS, etc
pg_config
pg_config.name
pg_config.setting
```

*** usage: get partitions of a table (declarative partition)**
Use **pg_inherits** for parent-child relationship.
Declarative partitions also use inheritance under the hoold

e.g.

```
SELECT
  parent.relname,
  partstrat as strategy,
  array(
    SELECT attname
    FROM UNNEST(partattrs) _(i)
    INNER JOIN pg_attribute ON partrelid = attrelid AND attnum = i
    ORDER BY array_position(partattrs, i) ASC
  ) as columns,
        -- partition key columns

        -- IMPORTANT: order matters
  array_agg(child.relname ORDER BY child.relname) as partitions
        --

-- get partitioned tables
FROM pg_partitioned_table
INNER JOIN pg_inherits ON inhparent = partrelid

-- get partitions of the table
INNER JOIN pg_class parent ON inhparent = parent.oid
INNER JOIN pg_class child ON inhrelid = child.oid
GROUP BY
  parent.oid, parent.relname, partstrat, partattrs
```

e.g. output

```
--[RECORD 1]--
table      | mytable
strategy   | r
columns    | {x}
partitions | {mytable_1, mytable_2, mytable_default}
```

==========

== Information schema==

information schema: https://www.postgresql.org/docs/current/information-schema.html

==========

```
This
is
code
Monaco 14
```